

Synchronous Design

Lecture 04

Josh Brake

Harvey Mudd College

Outline

- Review of basic synchronous design
- Review of the dynamic discipline
- Synchronizers and metastability
- FSM design steps
- **Activity:** design a level-to-pulse converter
- Diode and transistor review

Learning Objectives

By the end of this lecture you should be able to...

- Recall the dynamic discipline and timing specs for designing synchronous digital systems.
- Properly condition asynchronous signals using synchronizers.
- Design an FSM from a written spec: state transition diagram, table, and Verilog.
- Judge when a problem needs an FSM and when a few flops will do.
- Write a testbench that does not sample its own inputs on a clock edge.
- Recall how to use transistors to drive large currents.

Synchronous Digital Systems

- Timing problems are usually the #1 source of difficult bugs
- We can almost completely eliminate the timing problems with a synchronous discipline
 - Like digital vs. analog: digital is a subset of analog
 - Synchronous is a subset of asynchronous timing methodologies
 - Limiting choice makes design easier to understand and avoid sneaky bugs
- Also, will make testability easier (we'll see that later)

Basic Synchronous Design Rules

- Use only one _____ (named something clear like `clk`)
- Use only _____ as state elements (no latches!)
- Put this clock signal into the clock terminal of every flip-flop in the system.

Some common gotchas

Q: How do we begin in a known state?

A: _____

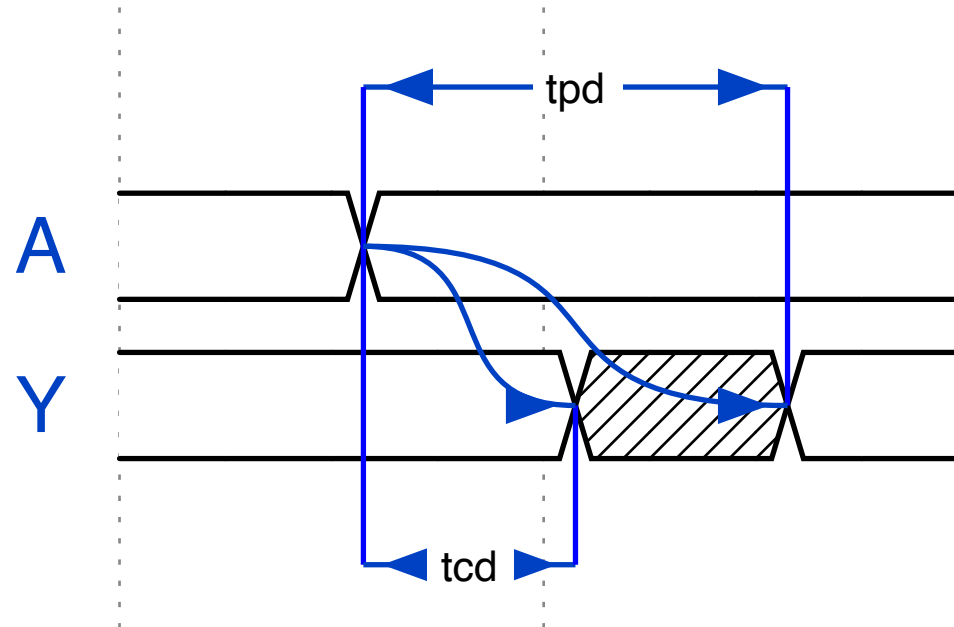
Q: How do we avoid changing the contents of a flip-flop on every clock cycle?

A: _____

Dynamic Discipline Review

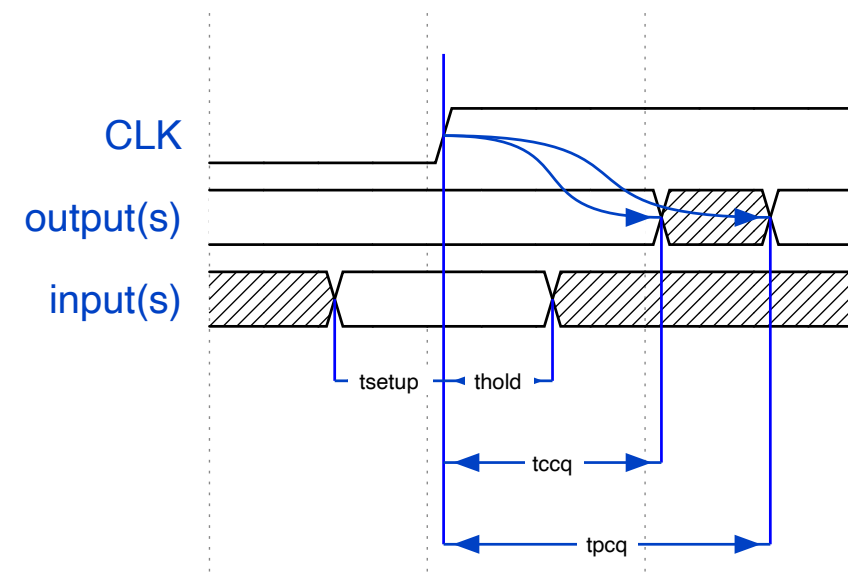
Propagation delay (t_{pd}) – the _____ time from when an input changes until the output(s) reach their final value.

Contamination delay (t_{cd}) – the _____ time from when an input changes until any output starts to change its value.



Dynamic Discipline Review: Sequential Logic

- **Propagation Clock-to-Q** (t_{pcq}) – _____ bound on the time from the rising edge of the clock until the output changes.
- **Contamination Clock-to-Q** (t_{ccq}) – _____ bound on the time from the rising edge of the clock until the output changes.
- **Setup time** (t_{setup}) – the amount of time an input to a flop must be stable _____ the clock edge.
- **Hold time** (t_{hold}) – the amount of time an input to a flop must be stable _____ the clock edge.

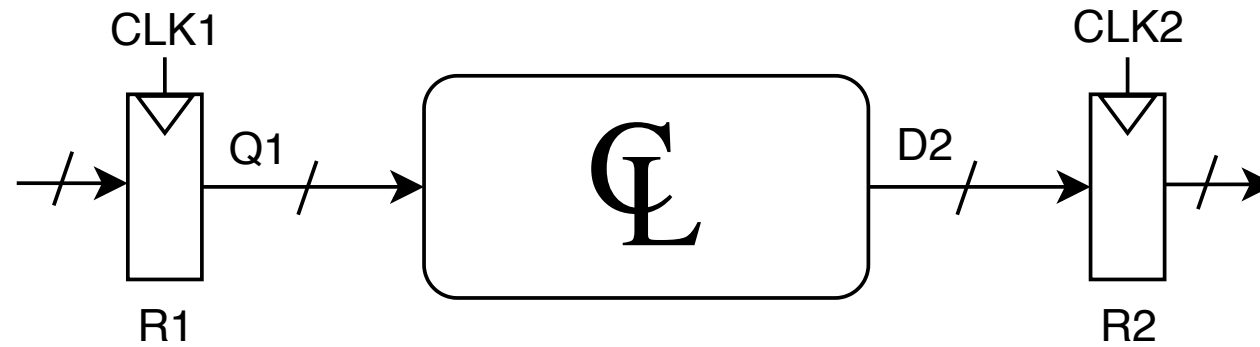


Synchronous Timing Constraints

Setup Time Constraint

Hold Time Constraint

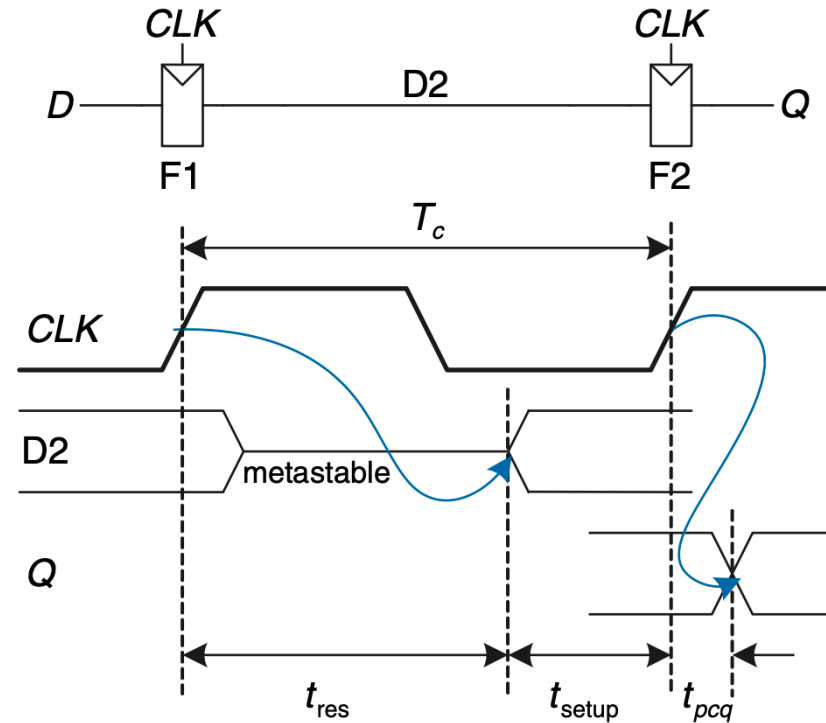
Understanding Timing Constraints



Synchronizers

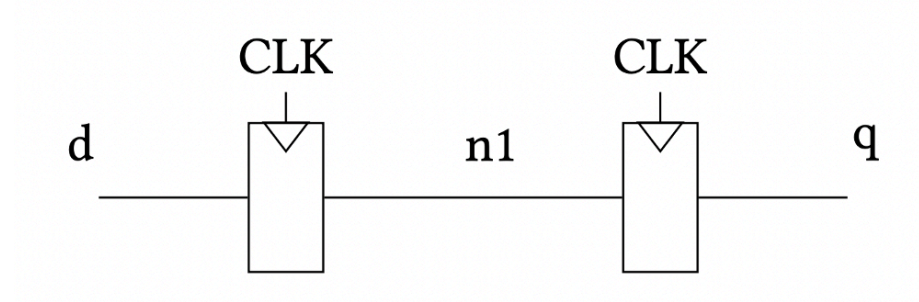
Synchronizer

- The world is asynchronous – how can we cope? Synchronizers!
- Simplest case is a 2-stage synchronizer made of two flops in series.
- If the output of flop F1 goes metastable, we have some time for it to resolve before the next clock edge and the second flop F2.
- This avoids passing metastable inputs out to combinational logic.



Synchronizer

```
1 module sync(input logic clk,  
2             input logic d,  
3             output logic q);  
4  
5     logic n1;  
6  
7     always_ff @(posedge clk)  
8     begin  
9         n1 <= d;  
10        q <= n1;  
11    end  
12 endmodule
```



Another Synchronizer (?)

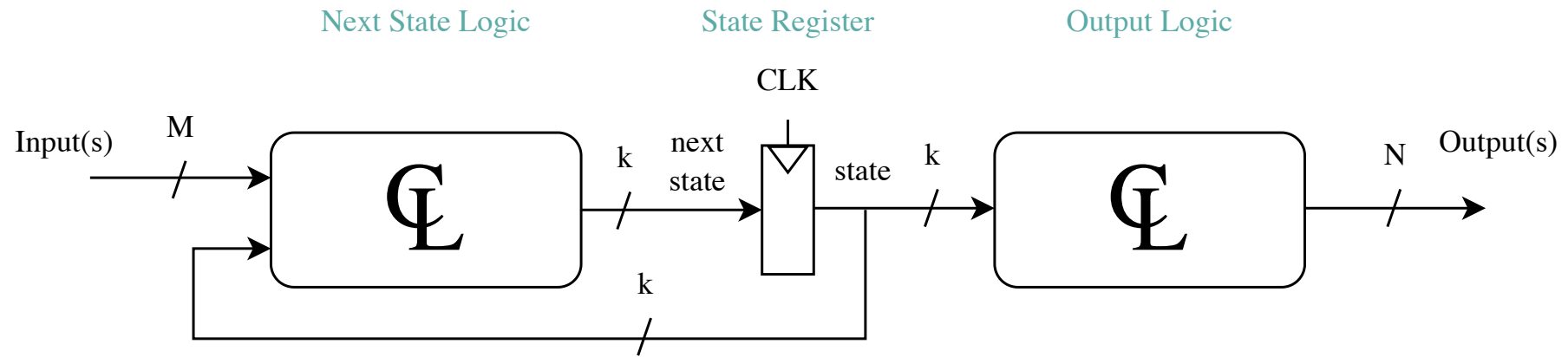
What if I replace the non-blocking assignments with blocking assignments?

What logic does this imply?

```
1  module sync(input  logic clk,  
2              input  logic d,  
3              output logic q);  
4  
5      logic n1;  
6  
7      always_ff @(posedge clk)  
8      begin  
9          n1 = d;  
10         q = n1;  
11     end  
12 endmodule
```

Finite State Machine (FSM) Review

FSM Design



FSM Design Process

1. List System Parameters (inputs & outputs)
2. Sketch State Transition Diagram
3. Write State Transition Table
4. Choose Encoding Scheme
5. Write Boolean Next State Equations
6. Write Output Table
7. Implement Logic

FSM Activity

Strobe Signal Generator (Level-to-pulse converter)

You have been tasked with creating circuitry for a single photon detector. When a photon arrives, it generates a pulse of a random length. We want to convert this random-length pulse to an output pulse of a fixed duration whenever a photon hits the detector.

Your task (should you choose to accept it): Design a system which generates a pulse for a single clock cycle when the input goes from low to high. Add a synchronizer to reduce the probability that the input will cause metastability.

How this is going to work

You are designing this one, not watching me design it.

Step	Time	Format
1. List inputs and outputs	2 min	on your own
2. Sketch the state transition diagram	5 min	in pairs
3. Two pairs put their diagram on the board	4 min	all of us
4. Write the Verilog	5 min	in pairs
5. Add the synchronizer the spec asked for	5 min	all of us
6. Find the bugs in my testbench	3 min	in pairs

Slides stay blank until we have talked about them. Resist the urge to look ahead.

Step 1: List the Specifications

On your own, 2 minutes. What goes in, what comes out?

Inputs

Outputs

Step 2: Sketch the State Transition Diagram

In pairs, 5 minutes. On your handout, draw the states and the transitions between them.

Some things to settle as you go:

- How many states do you need? Why can't you do it with fewer?
- Is **P** a function of the state alone, or of the state *and* **L**?
- What happens if **L** stays high for a hundred cycles?

Step 3: Put It on the Board

Two pairs, 4 minutes. Draw your diagram. We are looking for the *differences*.

Two Designs. Which One Meets the Spec?

3 states, Moore

```
assign P = (state == S1);
```

P depends on the state alone. Three states is the *minimum*: “idle” and “L is high, already pulsed” give the same output but behave differently, so they cannot be one state.

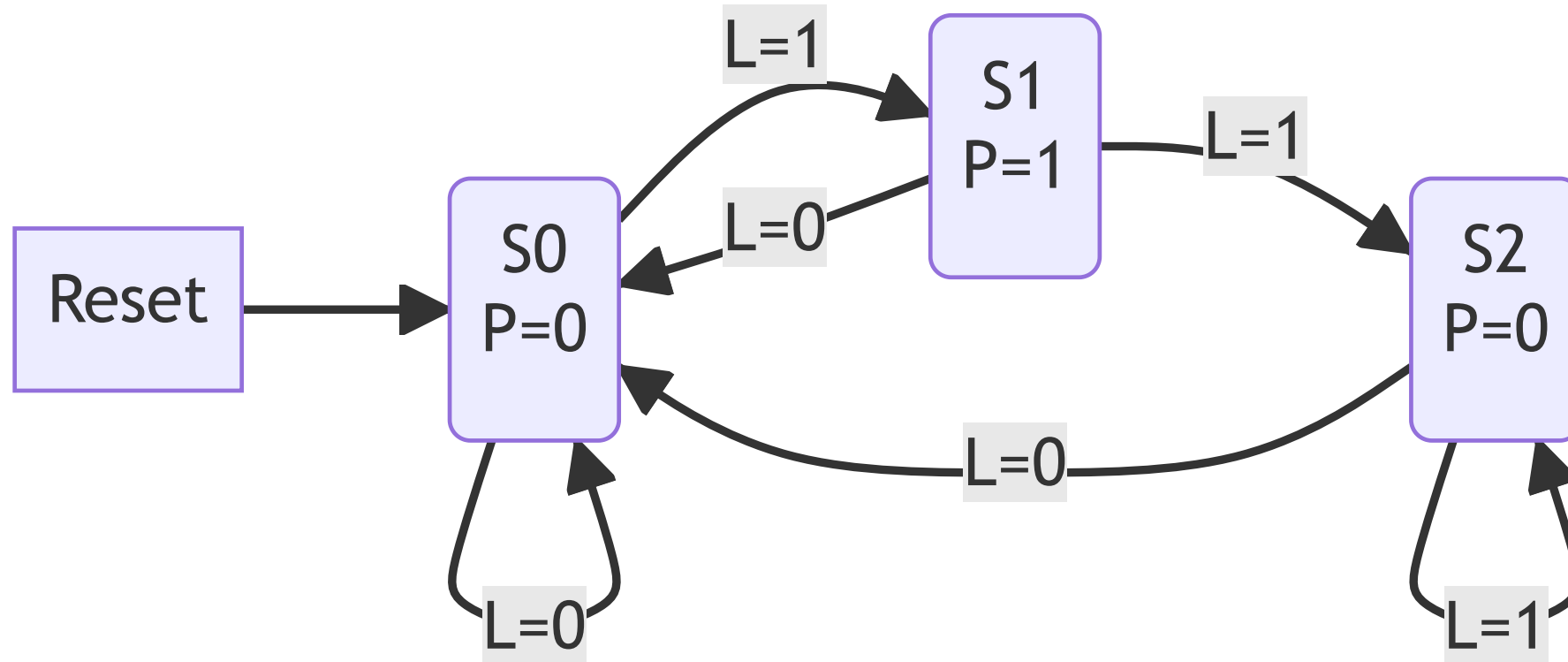
2 states, Mealy

```
assign P = (state == S0) & L;
```

One fewer flop, and **P** responds a cycle sooner.

The spec asked for a pulse of **fixed duration**. Does the Mealy version give you one?

State Transition Diagram



State Transition Table

Output Logic

Step 4: Write the Verilog

In pairs, 5 minutes. Five elements:

1. Inputs and outputs
2. Internal signal definition
3. State register: `always_ff` block. Make sure you have a reset!
4. Next state logic: `always_comb` block or `assign` statements.
5. Output logic: `always_comb` block or `assign` statements

FSM Verilog Template

Module and signal declaration.

```
1 // Produces a one clock cycle pulse on P each time L goes low to high.
2
3 module level_to_pulse_converter(
4     input  logic clk, reset,
5     input  logic L,
6     output logic P
7 );
8
9     logic [2:0] state, nextstate;
10    parameter S0 = 3'b001;
11    parameter S1 = 3'b010;
12    parameter S2 = 3'b100;
13
14    // Or, instead of the four lines above:
15    // typedef enum logic [1:0] {S0, S1, S2} statetype;
16    // statetype state, nextstate;
17    ...
```

FSM Verilog Template

State register.

```
1  ...
2      // State register
3      always_ff @(posedge clk, posedge reset)
4          if (reset) state <= S0;
5          else      state <= nextstate;
6
7  ...
```

Your Turn: Next State and Output Logic

Fill in the case body from your state transition table.

```
1      // Next state logic
2      always_comb
3          case (state)
4              S0:
5              S1:
6              S2:
7              default:
8          endcase
9
10     // Output logic
11     assign P =
```

What does the **default** case buy you here? Do you need it?

Next State and Output Logic

```
1  ...
2  // Next state logic
3  always_comb
4      case (state)
5          S0: if(L) nextstate = S1;
6              else nextstate = S0;
7          S1: if(L) nextstate = S2;
8              else nextstate = S0;
9          S2: if(L) nextstate = S2;
10             else nextstate = S0;
11         default: nextstate = S0;
12     endcase
13
14     // Output logic
15     assign P = (state == S1);
16 endmodule
```


Step 5: We Forgot Something

Reread the spec:

| Add a synchronizer to reduce the probability that the input will cause metastability.

Where does it go, and what does it cost you?

Tempting, But Wrong

The synchronizer's internal node `n1` is a one-cycle-delayed copy of `d`, and `q` is a two-cycle-delayed copy. So why not skip the FSM entirely?

```
1    // ... from the sync module a few slides back
2    always_ff @(posedge clk)
3        begin
4            n1 <= d;
5            q  <= n1;
6        end
7
8    assign P = n1 & ~q;    // one cycle wide when d rises. Ship it?
```

The Three-Flop Edge Detector

Synchronize first, *then* edge-detect. One more flop buys back the safety.

```
1    logic s1, s2, s3;
2
3    always_ff @(posedge clk, posedge reset)
4        if (reset) {s3, s2, s1} <= 3'b000;
5        else      {s3, s2, s1} <= {s2, s1, L};
6
7    assign P = s2 & ~s3;    // s2 is synchronized; s3 is s2 delayed one cycle
```

Same behavior as our FSM, no state encoding to get wrong.

Which One Would You Build?

FSM + synchronizer

- 5 flops
- ~30 lines
- Generalizes: add states as the spec grows

Three-flop edge detector

- 3 flops
- 3 lines
- Does exactly one thing

Count the logic cells each one costs. (We will do this properly next lecture.)

Writing a Testbench

Steps to create a testbench

1. Create clock signal which toggles continuously for any synchronous elements.
2. Initial statement to apply reset and set inputs to desired initial values.
3. Another initial block to apply input signals.

Don't apply signals on a clock edge! (e.g., if you are using a clock period of 10 timesteps, don't apply your inputs at multiples of 10.)

Step 6: Bug Hunt

In pairs, 3 minutes. Same game as last lecture – three things are wrong here.

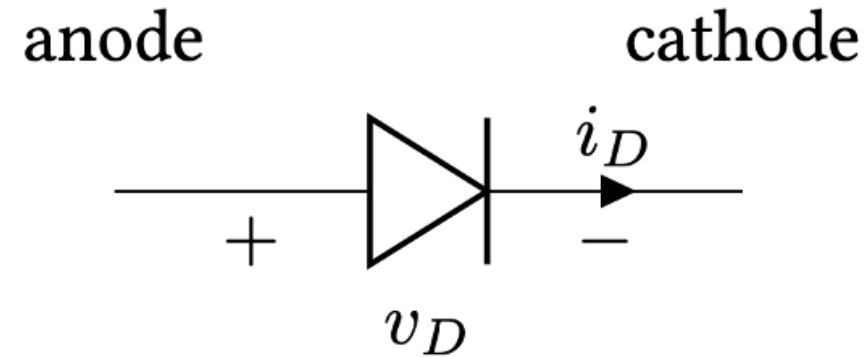
```
1 `timescale 1ns/1ns
2
3 module l2p_tb();
4     logic clk, reset;
5     logic L, P;
6
7     level_to_pulse_converter l2pc(clk, reset, L, P, debug);
8
9     always begin
10         clk = 1; #5; clk = 0; #5;
11     end
12
13     initial begin
14         reset = 1; #44; reset = 0;
15     end
16
17     initial begin
18         L = 0; #12; L = 1; #12;
19         L = 0; #44; L = 1; #20; L = 0; #32; L = 1;
20     end
21 endmodule
```

Testbench Code

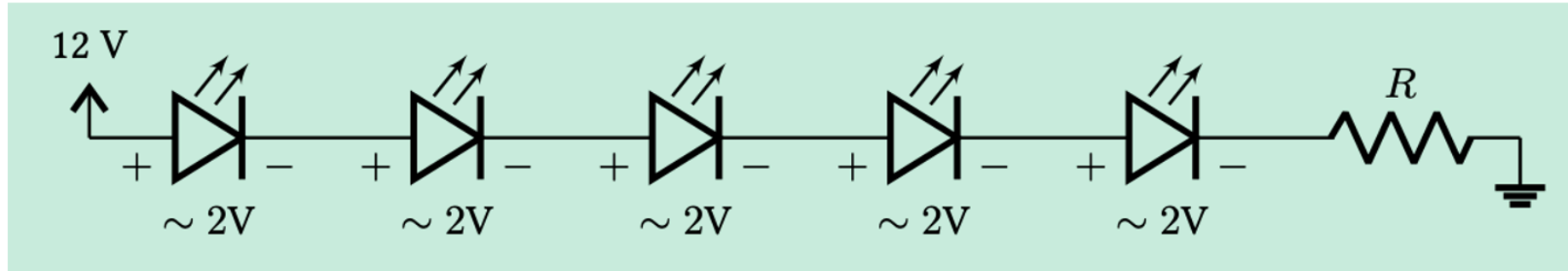
Testbench Code, cont.

Diode and Transistor Review

- $i_D = i_0 \left(\exp\left(\frac{v_D}{n \cdot v_T}\right) - 1 \right)$
- n and i_0 are scaling factors and v_T is the thermal voltage which is $v_T = kT/q$ (≈ 26 mV at room temperature).
- For a silicon diode, $v_{on} \approx 0.7V$ and for an LED $v_{on} \approx 1.7 - 2.1V$

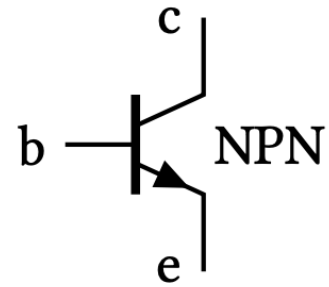


How many LEDs can you light up in series from a 12 V source?

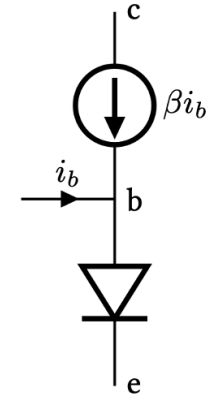


Transistors

Used to pull load _____. (i.e., connect load to _____)



NPN Symbol

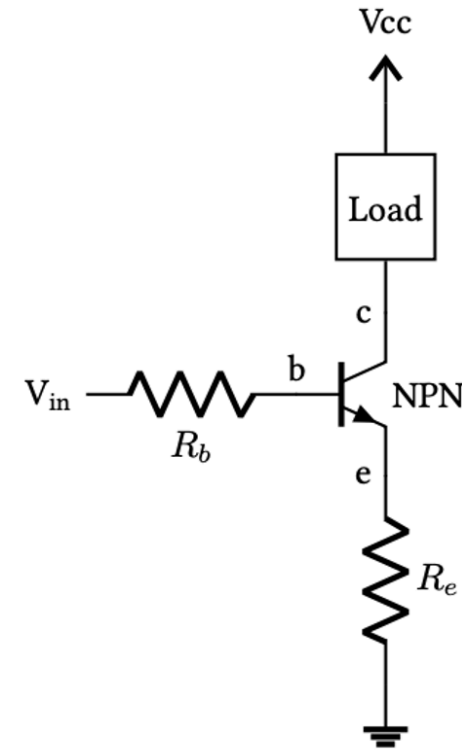


BJT small signal model

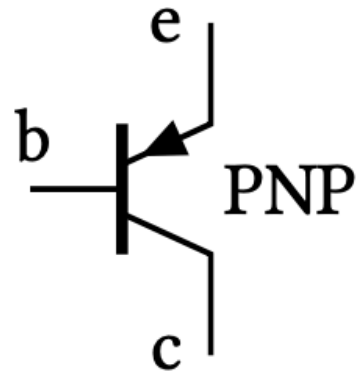
Driving a load with an NPN transistor

How do we choose R_b and R_e ?

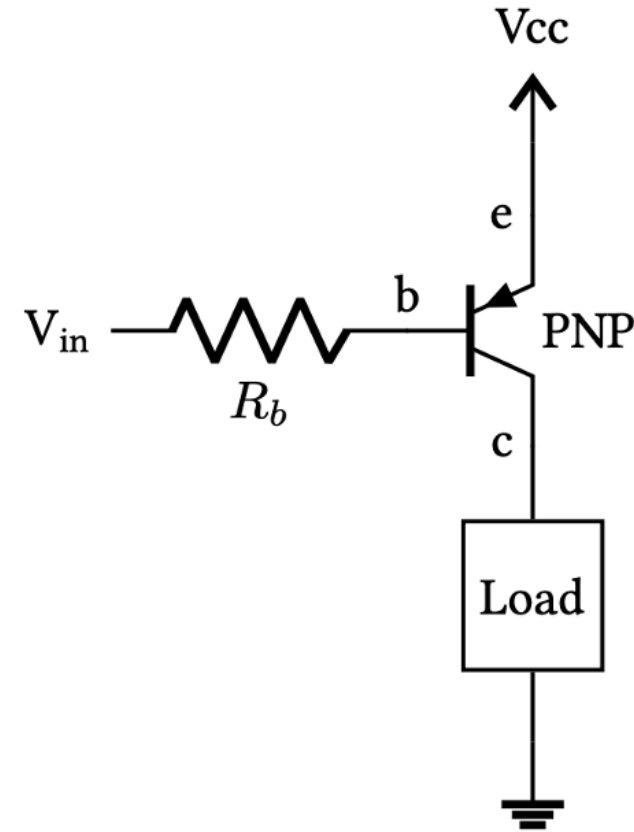
Set R_e to zero.



Driving a load with a PNP transistor



PNP Transistor Symbol



Used to pull load _____. (i.e., connect load to _____)

Wrap up

- Synchronous sequential design enables us to design simple and robust digital systems.
 - Only one clock signal to all flops (single clock domain)
 - Ensure that the setup and hold time constraints are observed.
- We need to synchronize asynchronous inputs to avoid metastability. Price is an additional clock cycle of latency.
- Transistors are like electrically controlled switches and enable us to drive larger loads from weak source (e.g., FPGA/MCU I/O pins)

Announcements/Reminders

- Checkoffs continue today – don't delay starting on Lab 2. Can reuse code from Lab 1
 - Only **one** seven_seg Verilog module.
 - Make sure LEDs are consistent brightness no matter how many segments are on
 - Develop a testbench to confirm your circuit is working. See tutorial on the website.